

Introduction to Claude Code

Francesco Caperna & Deli
July 10, 2026



Agenda

1. Introduction to the lecture

(Francesco)

- A gift for you...
- Why are we making this lecture?

2. Introduction to Claude Code

(Francesco)

- What is Claude Code?
- Git & getting started
- Context, commands & models

3. Tutorial: dataset analysis on a real dataset

(Deli)

4. Vibe coding exercise

Outline

1. Introduction to the lecture

2. Introduction to Claude Code

A gift for you...

We got you a **Claude Pro subscription**.

- Includes **Claude Code**
- Access to more capable Claude models
- More usage for everyday work
- you will use it for the project this week

Setup: follow the [installation guide](#)  on the course page.



Pro

For everyday productivity

\$17

Per month with annual subscription discount (\$200 billed up front). \$20 if billed monthly.

Try Claude

Everything in Free, plus:

- ✓ More usage*
- ✓ Includes Claude Code
- ✓ Includes Claude Cowork
- ✓ Includes Claude Design
- ✓ Includes Claude Science
- ✓ Access to unlimited projects to organize chats and documents
- ✓ Access to Research
- ✓ Ability to use more Claude models
- ✓ Claude for Microsoft 365
- ✓ Claude for Microsoft Outlook

Why are we making this lecture?

- It is important to know how to interact with Claude Code
- Projects (to implement a full pipeline)
- It does not solve all the problems, but it can help to explore different solutions quicker

Outline

1. Introduction to the lecture
2. Introduction to Claude Code

What is Claude Code?

Claude Code is an **AI coding assistant** that works directly in your terminal.

- It reads and understands your **entire codebase**
- You describe a task in **plain English**, and it plans and edits the code for you
- It can **run commands**: tests, builds, git, package installs, ...
- It comes with **different interfaces**: terminal, IDE (VS Code, JetBrains), desktop app

We focus on the **terminal interface: more control over your code, so you don't just vibe-code blindly.**

Why Git?

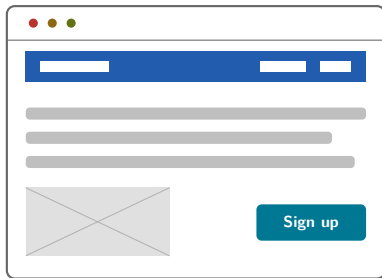
This week you will let **Claude Code** write code for you. Git is your safety net.

- **See what changed:** review every modification Claude makes to your files
- **Undo:** if a change breaks something, roll back to the last working version
- **Add features safely:** build each new feature as its own series of commits, keep what works

Commit before every big change: a **checkpoint you can always reload.**

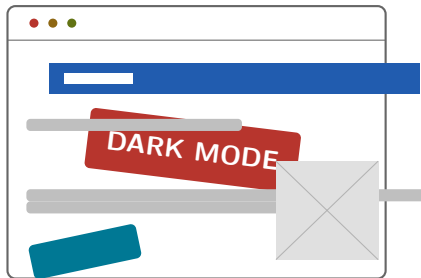
New feature, broken layout

You ask Claude for a small change: “add a dark mode button”. It edits your files, and...



before: everything works

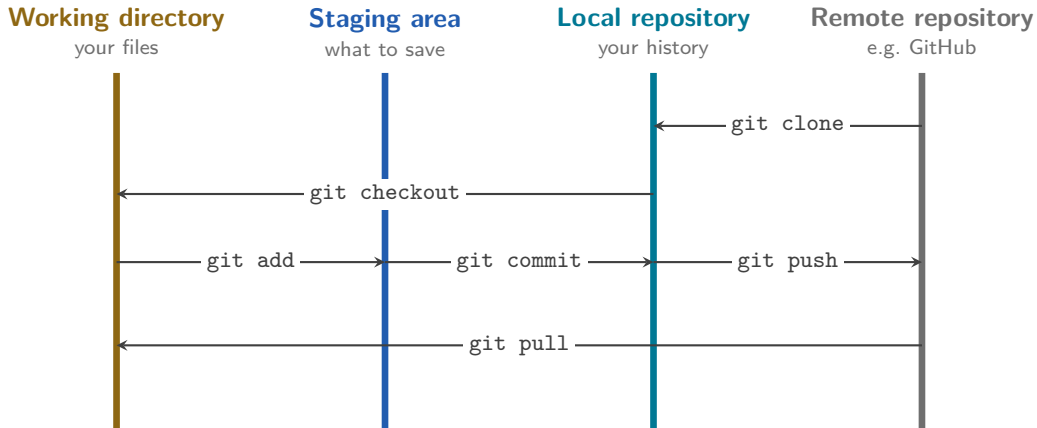
new button
→



after: the layout falls apart

Without Git you undo this by hand. With Git it is one command back to the [working version](#).

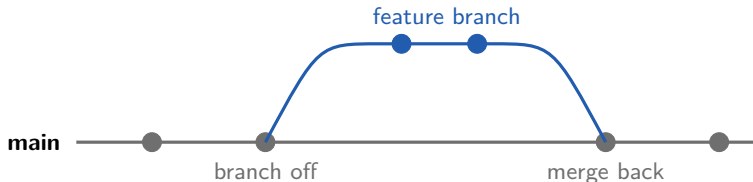
How Git works: the four areas



Daily loop: `git add`, then `git commit`. **Push and pull** only to sync with GitHub.

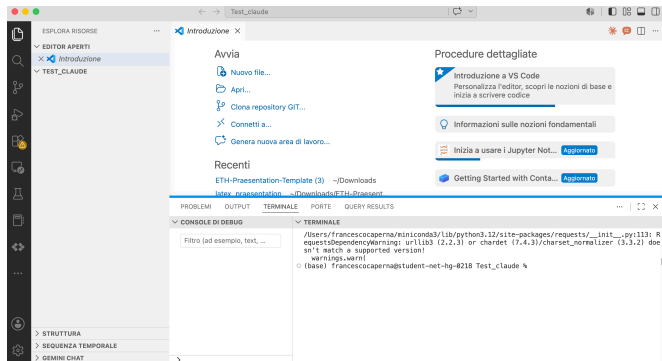
Branches: a tree of commits

A **branch** is a parallel line of commits: experiment freely in isolation, then **merge** back. If it goes wrong, **main** stays safe.



Ask Claude Code to work on a **branch:** review its commits, merge them into **main** if you like them, or simply throw the branch away.

Step 1: Open VS Code and its terminal



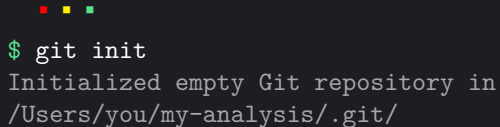
Open your project folder in **VS Code**.

Then open the built-in terminal:

- Menu **Terminal** → **New Terminal**
- or press **Ctrl + `**

You will run all the next commands right here, inside VS Code.

Step 2: Turn it into a Git repository



```
$ git init  
Initialized empty Git repository in  
/Users/you/my-analysis/.git/
```

`git init` creates the hidden `.git` folder. Your project is now tracked by Git.

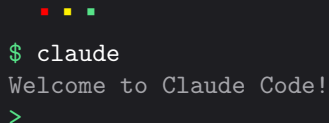
Step 3: Save your first snapshot

```

$ git add .
$ git commit -m "Initial commit"
[main (root-commit) a1b2c3d] Initial commit
```

`git add` stages your files, `git commit` saves a snapshot you can always go back to.

Step 4: Start Claude Code

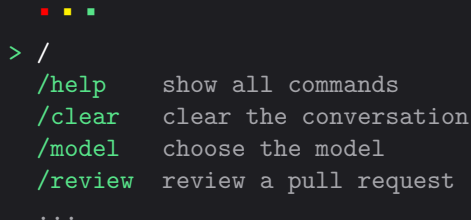
A dark-themed terminal window with three colored squares (red, yellow, green) in the top left corner. It shows a green prompt character followed by the command 'claude', the output 'Welcome to Claude Code!', and a green prompt character on the next line.

```
$ claude
Welcome to Claude Code!
>
```

Now launch **Claude Code** inside the repo, every change it makes is tracked by Git.

Type / to access commands

In Claude Code, type `/` to open the command menu. Keep typing to filter, then hit enter.

A screenshot of the Claude Code command menu. At the top left, there are three small colored squares (red, yellow, green). Below them, a green prompt character '>' is followed by a forward slash '/'. A list of commands follows, each with a green prefix and a description in white text. The commands are: '/help' (show all commands), '/clear' (clear the conversation), '/model' (choose the model), and '/review' (review a pull request). The list ends with three dots '...' on a new line.

```
> /  
/help    show all commands  
/clear   clear the conversation  
/model   choose the model  
/review  review a pull request  
...
```


How to give Claude context

The more context Claude has, the better it works. You can provide it in several ways.

- **CLAUDE.md**: permanent context, loaded automatically every session
- **The prompt**: describe your task in plain English, the more specific the better
- **Point to files**: name the exact files Claude should look at
- **Images**: drag in a screenshot (an error, a mockup, a chart)
- **Command output**: Claude reads the output of the tests and builds it runs, so it learns from errors

Good context in, good code out.

CLAUDE.md: give Claude project context

A CLAUDE.md file in your project is read **automatically** at the start of every session. Think of it as Claude Code's memory.

What to put in it

- How to build, test, and run the project
- **The project structure** and key files
- Coding conventions and style
- Things to avoid

Tip: run `/init` to generate a first version automatically.

```
# CLAUDE.md

## Commands
- Run tests: pytest
- Start app: python main.py

## Project structure
- src/      core code
- data/     raw datasets (do not edit)
- tests/    unit tests

## Conventions
- Use type hints everywhere
```

/init: bootstrap your CLAUDE.md



```
> /init  
Analyzing your project...  
Created CLAUDE.md
```

/init scans your project and writes a first CLAUDE.md for you.

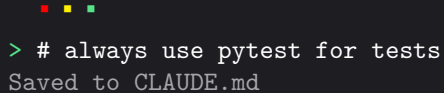
@: point Claude to a file



```
> Explain what @src/main.py does
```

Type @ followed by a path to pull a specific file into the context.

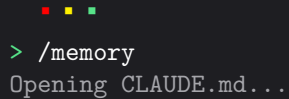
#: add a quick memory



```
> # always use pytest for tests  
Saved to CLAUDE.md
```

Start a message with # to append a note to CLAUDE.md on the fly.

/memory: edit project memory



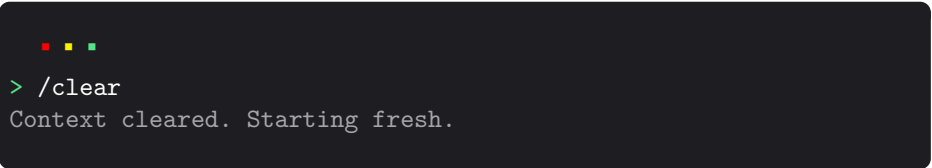
```
> /memory  
Opening CLAUDE.md...
```

Opens your memory files so you can review or edit them.

Why a good context matters

Claude works best with a **focused, relevant** context. Too much unrelated information makes it slower and less accurate, so keep only what the current task needs.

When you switch to a different task, start fresh:

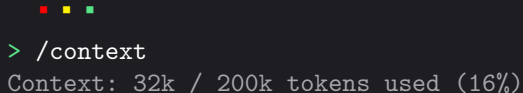


```
> /clear  
Context cleared. Starting fresh.
```

Mind your token usage

The context window is **limited** and measured in tokens. Your code, the open files and the whole conversation all count. When it fills up, Claude can lose track of earlier details.

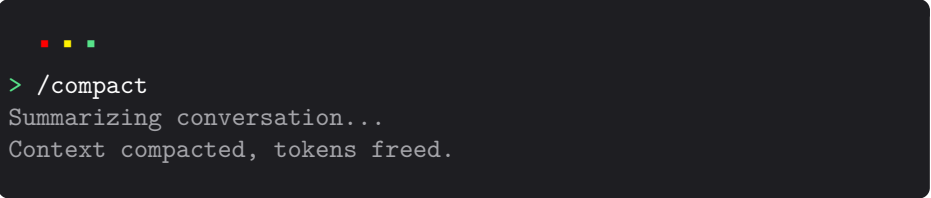
Check how much you are using:

A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. The prompt is a green greater-than sign. The command entered is `/context`. The output is `Context: 32k / 200k tokens used (16%)`.

```
> /context
Context: 32k / 200k tokens used (16%)
```


`/compact`: free up the context

In a long session the conversation keeps growing and eats tokens. `/compact` summarizes what happened so far, keeping the important points while freeing space.



```
> /compact
Summarizing conversation...
Context compacted, tokens freed.
```

Choosing the model

Pick the model based on the task: more capable models reason better but are slower and cost more.

- **Opus**: most capable, for complex or tricky tasks
- **Sonnet**: fast and balanced, a good default
- **Haiku**: quickest and cheapest, for simple tasks

```

■ ■ ■
> /model
Opus      most capable
Sonnet    fast and balanced (default)
Haiku     quickest
```

Plan, Think, Edit: three ways to work

Plan

Claude researches and proposes a step by step plan. No changes until you approve.

Best for complex or risky tasks.

Think

Claude reasons more deeply before acting, using extra thinking budget.

Best for tricky bugs and design decisions.

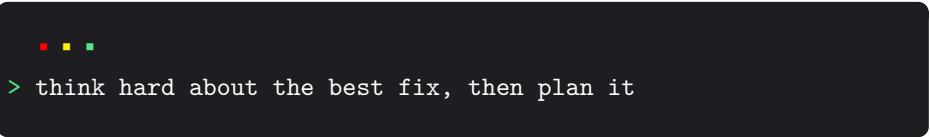
Edit

Claude reads and edits your files directly to implement the task.

Best for clear, well defined tasks.

Switching between modes

- Press Shift + Tab to cycle: **Edit** → auto-accept → **Plan**
- For deeper reasoning, just ask Claude to **think** in your prompt



```
> think hard about the best fix, then plan it
```

The stronger the wording (think, think hard, ultrathink), the more Claude reasons before answering.

Create your own slash command

You can define custom commands. Put a Markdown file in `.claude/commands/`, and it becomes a command you can call.

```
.claude/commands/test.md
```

Run the test suite and fix
any failing tests.

```
> /test  
Running tests...
```

Great for tasks you repeat often.

Handy slash commands

- `/help`: list every command
- `/clear`: reset the conversation
- `/compact`: summarize to free tokens
- `/context`: show token usage
- `/model`: switch model
- `/init`: create CLAUDE.md
- `/memory`: edit memory
- `/review`: review a pull request
- `/agents`: manage subagents
- `/config`: open settings

Not sure? Just type `/help`.

Don't memorize, just ask

You don't need to remember every command by heart. What matters is **knowing these features exist**.

- Forgot a command? Just ask Claude in plain English
- Claude will tell you the right command for what you want

```
> how do I free up the context?  
Use /compact to summarize the conversation.
```

Subagents (*optional*)

For big tasks, Claude can delegate parts of the work to specialized **subagents** that run on their own.

- Each subagent has its own focused context and task
- Useful for large searches, multi step refactors, or reviews
- Manage them with `/agents`

```
> /agents  
Manage your subagents...
```

Nice to know, not needed for today.

MCP (*optional*)

MCP (Model Context Protocol) lets Claude Code connect to **external tools and data sources**.

- Examples: databases, GitHub, Slack, a web browser, ...
- Claude can then read from and act on these tools directly
- Add servers with `claude mcp add`, inspect them with `/mcp`

```
> /mcp  
Manage MCP servers...
```

Nice to know, not needed for today.

Thank you for your attention!

Questions?

Want to go deeper? [Claude Code course](#)  by Net Ninja on YouTube.